Confident Agent: Progress Report, Stage 1

Benchmark Construction and Baseline Evaluation

COMP301 Software Design (2026). Group: Yakhani Dlamini, Simbonge Ndlovu, Mfundo Magasela

1. Summary of Progress

We have completed Stage 1 of the project: the application domain is fixed, a 76-prompt hallucination benchmark has been built and documented, and a baseline evaluation has been run and scored. The baseline is a single-prompt LLM tutor (llama3.2:3b, run locally through Ollama) with no validation pipeline.

Key finding: the baseline produced a hallucination in 37 of 76 prompts (48.7%, 95% Wilson interval 37.8% to 59.7%). The hallucination rate rises sharply with prompt obscurity (18.8% at level 1, 46.9% at level 2, 67.9% at level 3), which supports the benchmark's design assumption and leaves substantial room to measure the effect of the Confident Agent.

This stage supersedes the preliminary 20-prompt pilot we reported earlier.

2. Changes Since the Pilot

	Pilot (earlier report)	Stage 1 (this report)
Model	DeepSeek V4 Pro (chat interface)	llama3.2:3b (local Ollama)
Prompts	20	76
Categories	5 (no materials)	6 (learning materials added)
Baseline hallucination rate	10.0% (2/20)	48.7% (37/76)

We made three changes, each for a methodological reason. First, a 10% baseline error rate leaves too little room to detect an improvement, so we moved to a smaller local model that fails more often. Second, the pilot did not cover fabricated learning materials, one of the

three hallucination types in our proposal, so a materials category was added. Third, a locally hosted model has a fixed version (limiting model drift), is reproducible, and can be used unchanged as the generator inside the Confident Agent, so the baseline and the intervention share the same underlying LLM. The DeepSeek pilot results are retained as supporting evidence.

3. Application Domain and Scope

The domain is educational tutoring for two first and second-year Computer Science topic areas: Java object-oriented programming (COMP200) and software design (COMP301). The system generates lesson explanations, quiz questions with answer keys, and learning-material recommendations. COMP102 and COMP201 were considered and deliberately excluded to keep the curated reference set bounded; they are noted as future work.

4. Benchmark Design

The benchmark contains 76 prompts. Each has an ID, category, course, topic, obscurity level (1 to 3), the prompt text, a written ground truth, and a scoring method. Ground truth was written before the baseline was run, and the prompts have not been changed since.

Category	Prompts	What it tests	Scoring
A. Lesson content	18	Correctness of concept explanations and Java examples	Manual, against ground-truth claims
B. Obscure knowledge	10	Specific terminology and standard-library facts	Manual
C. Quiz answer keys	15	Correct option or true/false verdict	Answer extraction, checked manually
D. Code reasoning	8	Predicting output and diagnosing design flaws in code	Manual
E. Hallucination traps	10	Fabricated concepts or APIs and false premises	Manual: did the model challenge the premise?
F. Learning materials	15	Existence and attribution of books, papers and links	Lookup against bibliographic sources

Obscurity levels were assigned when the benchmark was constructed: level 1 covers core concepts taught in lectures, level 2 covers standard material with commonly confused details, and level 3 covers obscure terminology, fine-grained API facts and fabricated premises. The split by course is 24 COMP200 prompts and 52 COMP301 prompts. The 20 pilot prompts were retained; one (the Delegation pattern prompt) was reworded to remove an ambiguous attribution.

5. Baseline Setup

Every prompt was sent independently to llama3.2:3b with the same system prompt as the pilot: act as a university-level tutor, answer directly, do not assume information in the question is correct, and state uncertainty rather than invent facts. No validation, retrieval or retry was used.

- **Settings:** temperature 0, seed 42, maximum 600 generated tokens, so the run is deterministic and reproducible.
- **Hardware:** a UKZN lab PC (Intel i7-7700, 16 GB RAM) in CPU-only mode. GPU inference was disabled because the 2 GB graphics card produced corrupted output with this model.

- **Runtime:** responses took roughly 5 to 65 seconds each, with long lesson explanations at the upper end.
- **Truncation:** 8 of the 76 responses were cut off by the 600-token cap. Truncated answers were scored only on what was actually written. The same cap will be used for the Confident Agent.

6. Scoring Procedure

Responses were scored by the group against the written ground truth.

- **Correct** means every required ground-truth point is present and nothing false is stated.
- **Hallucination** means the response contains a false or invented statement, a wrong answer key, or accepts a false premise. For quiz-key prompts (category C), hallucination is defined as a wrong answer key. A response can therefore be incorrect without hallucinating, for example when it abstains or omits a required point.
- Quiz keys were extracted automatically and checked by hand. Code-reasoning prompts were checked against the definitions and by running the code. Learning materials were checked by looking up each title, author and year.
- After a consistency review, five rows were re-scored to apply the rubric uniformly (A02, E04, E09, F12, F13). Five borderline rows (A05, A08, A17, B05, D06) are being decided by a second scorer, and a further independent re-scoring of a sample of rows will be used to report scorer agreement.

7. Baseline Results

Metric	Value
Total prompts	76
Hallucinated responses	37
Responses without a hallucination	39
Baseline hallucination rate (HR_baseline)	**48.7%**
95% confidence interval (Wilson)	37.8% to 59.7%
False rejection rate (FRR)	N/A (no validation pipeline)

By category

Category	Hallucinated / Total	Rate
A. Lesson content	8 / 18	44.4%
B. Obscure knowledge	8 / 10	80.0%
C. Quiz answer keys	2 / 15	13.3%
D. Code reasoning	4 / 8	50.0%
E. Hallucination traps	8 / 10	80.0%
F. Learning materials	7 / 15	46.7%

By obscurity level

Obscurity	Hallucinated / Total	Rate
1 (core)	3 / 16	18.8%
2 (standard, easily confused)	15 / 32	46.9%
3 (obscure or fabricated)	19 / 28	67.9%

By course: COMP200 14 / 24 (58.3%); COMP301 23 / 52 (44.2%). This difference is confounded with the mix of categories and obscurity levels in each course, so we do not draw conclusions from it.

8. Observations

1. **The weak baseline gives measurable headroom.** The pilot's 10% rate would have made any improvement hard to detect. At 48.7%, a paired test on the same 76 prompts has meaningful power.
2. **Obscurity predicts hallucination.** The rate rises from 18.8% to 67.9% across the three levels, consistent with our hypothesis and with the pilot's finding that fabricated or obscure premises are the main failure point.
3. **Quiz keys were mostly correct, but the errors were serious.** Only 2 of 15 keys were wrong, yet one selected the Open/Closed Principle for a Dependency Inversion question, and another judged a true statement false and cited an invented 1965 thesis

as support.

4. **The model often failed to challenge false premises.** It accepted or elaborated on a fabricated interface (E02), a fabricated theorem (E03), a mutable-String premise (E06), a Java `async` keyword (E08) and a fabricated queue class (E10). Even when it rejected the premise it sometimes added false supporting detail (E04, E09).
5. **Code examples frequently failed to illustrate the concept.** In several lesson answers (for example A02, A03, A07 and A11) the definition was acceptable but the Java example contradicted it, such as a "tight coupling" example that already used constructor injection.
6. **Fabricated references appeared in materials.** The model invented a book (a Kent Beck title on refactoring), attached wrong years or authors to real books, and produced unverifiable paper and arXiv citations (F02 to F06, F14). It also abstained on several prompts (F07, F10, F11, F12), which we scored as non-hallucinations.

9. Preliminary Threats to Validity

- **Weak baseline:** a 3B model may exaggerate the benefit of validation; the effect may be smaller for stronger models.
- **Evaluator bias:** ground truth and scoring were produced by the group. A second scorer and an agreement measure will mitigate this.
- **Subjective obscurity labels:** levels were assigned at construction time rather than measured.
- **Truncation and single run:** the 600-token cap cut off 8 responses, and a deterministic single run gives no estimate of run-to-run variance.
- **Benchmark size:** with 76 prompts the confidence intervals are wide (about plus or minus 11 percentage points).
- **Reference-set independence:** the Confident Agent's reference notes will be written from lecture material at topic level, not from the benchmark ground truth, to avoid circular evaluation.

10. Next Steps

Confident Agent design. The generator will be the same llama3.2:3b with the same system prompt and settings. The baseline response is reused as the agent's first candidate, so the comparison is paired prompt by prompt. A Verifier built on a smaller model from a different family will check candidates: it re-solves quiz questions, compares

lesson and code answers against topic-level reference notes (returning supported, contradicted or unsure), and tests learning materials with bibliographic API lookups. A Confidence Estimator will turn verifier agreement and consensus voting into a score, and the Chief Agent will accept, regenerate once with feedback, or flag the content as low confidence.

Evaluation. The identical 76 prompts will be run through the agent without modification. We will report:

- HR_agent and Hallucination Reduction = (HR_baseline − HR_agent) / HR_baseline × 100
- False Rejection Rate (FRR) = correct answers incorrectly flagged as untrustworthy / total correct answers × 100
- McNemar's test on the paired outcomes, with confidence intervals
- latency, number of model calls and other cost overheads

Remaining milestones: write the topic reference notes, implement the agent, run the comparative evaluation, complete the statistical analysis, and write the final report.

11. Use of Generative AI

Anthropic's Claude was used to help draft benchmark prompts and ground truths, to write the benchmark runner and scoring scripts, and to help structure this report. The group reviewed the benchmark, verified and scored the responses, and is responsible for the content.

Appendix: Artefacts

- `benchmark.csv` : the 76 benchmark prompts with ground truth, course, topic, obscurity and scoring method
- `run_baseline.py` : baseline runner (fixed settings, CPU mode)
- `score_auto.py` : quiz-key scorer and truncation flag
- `results_baseline_llama3.2_3b_scored.csv` : raw responses and manual scores